C Programming Language Evaluation Report

C is highly capable of implementing the core functionality of the Banking and Currency Exchange Application, but it requires a more manual, procedural approach compared to modern languages. Unlike Kotlin or R, which offer built-in classes or hash maps, C relies on custom structs to group related data and fixed-size arrays to manage lists. In this implementation, a BankAccount struct holds the user's name and balance in memory, while basic parallel arrays track currency names and exchange rates. Custom struct to manage state, Codeblock :

```c

// Structure to track users active banking session
typedef struct {
    char accountName[100];
    double balance;
    int isRegistered; // 0 for unregistered, 1 for registered
} BankAccount;

// Array storing the exchange rates, PHP is always 1.00 at 0 index
// Indices are 0 = PHP, 1 = USD, 2 = JPY, 3 = GBP, 4 = EUR, 5 = CNY
double exchangeRates[6] = {1.00, 62.00, 0.40, 84.00, 72.00, 9.00};
```

II. Expressiveness and Readability

C is a low-level language, making its syntax explicit and very technical. It lacks easy expression like you would find in Kotlin (like string templates or type inference) and R. Variables must be strictly typed and declared before use. Additionally, interacting with standard input requires explicit formatting specifiers (like %lf for doubles) and passing memory addresses using the & operator. While this explicitness gives the programmer a better amount of control over what the machine is doing, it can hinder readability for developers accustomed to higher-level abstractions, as the code often focuses heavily on memory management rather than pure business logic. Pointers and addresses are needed, Codeblock:

```c
    printf("Withdraw Amount: ");
    scanf("%lf", &withdrawAmount);

    if (withdrawAmount > 0 && withdrawAmount <= acc->balance) {
        acc->balance -= withdrawAmount; // Subtract from struct state
        printf("Updated Balance: %.2f\n", acc->balance);
    } else if (withdrawAmount > acc->balance) {
        printf("Insufficient balance.\n");
    } else {
        printf("Invalid withdraw amount.\n");
    }
```

III. Reliability and Safety

Reliability is C's biggest weakness. Unlike Kotlin (which has null safety) or R (which has tryCatch for runtime errors), C assumes the programmer is not making any mistakes. There is no automatic memory management, and C does not natively throw catchable exceptions when things go wrong. For example, if a user inputs a letter instead of a number during the scanf prompt for a deposit, the standard input buffer breaks, causing an infinite loop. To make the application reliable, manual safeguards are needed like flushing the input buffer character by character. C trusts the developer to handle buffer overflows and type mismatches manually, which increases the risk of critical runtime failures. Code block showing how to do it:

```
// Ensures scanf works properly
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) { }
}
```

IV. Tooling and Ecosystem

C benefits from a universally available ecosystem, but it lacks the modern application tooling seen in newer languages. Compilers like GCC or Clang are complex, and debugging is handled via tools like GDB. However, C has no standard package manager (unlike R's CRAN or Kotlin's Gradle/Maven). Adding external libraries in C is tedious and requires more know how to operate with and all is manually done unlike newer languages today.

V. Performance and Scalability

C is still one of the fastest languages today. Because it compiles directly to native machine code without the additional resources used by a Virtual Machine (like Kotlin's JVM) or an interpreter (like R), the application runs instantaneously. For a single-user console application, C's performance is very great. However, scaling this specific implementation to support thousands of concurrent banking transactions would be highly complex.

VI. Comparison and Reflection

Compared to Kotlin, Java and R, C is a fundamentally different tool. Kotlin and Java utilizes object-oriented paradigms to encapsulate state cleanly and provides robust null-safety. R excels in data structures and built-in serialization for rapid data manipulation. C, by contrast, is entirely procedural and manual. While C easily meets the core arithmetic and menu-routing

requirements of this banking application, it forces the developer to write boilerplate code just to safely handle basic keyboard input and memory addressing.

C is a fast language however its complexity and scalability issues are hard to handle and the benefits it brings don't outweigh the troubles it brings, for a program like the Banking, C’s speed is overkill.